



Modelo Numérico da Atmosfera

MET-576-4

Modelagem Numérica da Atmosfera

Dr. Paulo Yoshio Kubota

Os métodos numéricos, formulação e parametrizações utilizados nos modelos atmosféricos serão descritos em detalhe.

**3 Meses
24 Aulas (2 horas cada)**



- ✓ **Métodos de diferenças finitas.**
- ✓ **Acurácia.**
- ✓ **Consistência.**
- ✓ **Estabilidade.**
- ✓ **Convergência.**
- ✓ **Grades de Arakawa A, B, C e E.**
- ✓ **Domínio de influência e domínio de dependência.**
- ✓ **Dispersão numérica e dissipação.**
- ✓ **Definição de filtros monótono e positivo.**
- ✓ **Métodos espectrais.**
- ✓ **Métodos de volume finito.**
- ✓ **Métodos Semi-Lagrangeanos.**
- ✓ **Conservação de massa local.**
- ✓ **Esquemas explícitos versus semi-implícitos.**
- ✓ **Métodos semi-implícitos.**



O Esquema CTCS (Leapfrog)



Um outro método para resolver o problema de advecção linear. É usar o esquema centrada no tempo, Centrado no espaço (CTCS). i.e.

$$\frac{\phi_j^{n+1} - \phi_j^{n-1}}{2\Delta t} + u \frac{\phi_{j+1}^n - \phi_{j-1}^n}{2\Delta x} = 0. \quad (17)$$

Trata-se de uma fórmula de três-nível, uma vez que ela envolve valores de ϕ em três tempos t_{n+1} , t_n , t_{n-1} .

O esquema CTCS é de precisão de segunda ordem no espaço e no tempo. análise de estabilidade Von Neumann

Define-se $\phi = A^n e^{ikj\Delta x}$ para a análise de **estabilidade de Von Neumann**, que veremos em seguida

$$\begin{aligned} \phi_j^{n+1} &= \phi_j^{n-1} - u \frac{\Delta t}{\Delta x} (\phi_{j+1}^n - \phi_{j-1}^n) \\ A^{n+1} e^{ikj\Delta x} &= A^{n-1} e^{ikj\Delta x} - u \frac{\Delta t}{\Delta x} (A^n e^{ik(j+1)\Delta x} - A^n e^{ik(j-1)\Delta x}) \end{aligned}$$



O Esquema CTCS (Leapfrog)

$$A^{n+1}e^{ikj\Delta x} = A^{n-1}e^{ikj\Delta x} - u \frac{\Delta t}{\Delta x} (A^n e^{ik(j+1)\Delta x} - A^n e^{ik(j-1)\Delta x})$$

$$A^n A e^{ikj\Delta x} = \frac{A^n}{A} e^{ikj\Delta x} - u \frac{\Delta t}{\Delta x} (A^n e^{ikj\Delta x} e^{ik\Delta x} - A^n e^{ikj\Delta x} e^{-ik\Delta x})$$

$$A^n A e^{ikj\Delta x} = \frac{A^n}{A} e^{ikj\Delta x} - C (A^n e^{ikj\Delta x} e^{ik\Delta x} - A^n e^{ikj\Delta x} e^{-ik\Delta x})$$

$$A^n A e^{ikj\Delta x} = \frac{A^n}{A} e^{ikj\Delta x} - C A^n e^{ikj\Delta x} (e^{ik\Delta x} - e^{-ik\Delta x})$$

$$A = \frac{1}{A} - C (e^{ik\Delta x} - e^{-ik\Delta x})$$

$$A^2 = 1 - C 2i \left(\frac{e^{ik\Delta x} - e^{-ik\Delta x}}{2i} \right) A$$

$$A^2 = 1 - C 2i (\sin(k\Delta x)) A$$



$$A^2 = 1 - C2i(\sin(k\Delta x))A$$

$$A^2 + C2i(\sin(k\Delta x))A - 1 = 0$$

$$A^2 - C2i(\sin(k\Delta x))A - 1 = 0$$

$$\begin{cases} ax^2 + bx + c = 0 \\ x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \end{cases}$$

$$A = \frac{C2i(\sin(k\Delta x)) \pm \sqrt{(-C2i(\sin(k\Delta x)))^2 - 4(1)(-1)}}{2}$$

$$A = \frac{C2i(\sin(k\Delta x)) \pm \sqrt{((-1)^2 C^2 4(\sqrt{-1}^2)(\sin^2(k\Delta x)))^2 + 4}}{2}$$

$$A = \frac{C2i(\sin(k\Delta x)) \pm \sqrt{-4C^2 \sin^2(k\Delta x) + 4}}{2}$$



$$A = Ci(\sin(k\Delta x)) \pm \sqrt{-C^2 \sin^2(k\Delta x) + 1}$$

$$A = Ci(\sin(k\Delta x)) \pm \sqrt{1 - C^2 \sin^2(k\Delta x)}$$

Há dois casos a considerar: o primeiro $C > 1$, e $(C \sin(k\Delta x))^2 > 1$

$$A = Ci(\sin(k\Delta x)) \pm i\sqrt{C^2 \sin^2(k\Delta x) - 1}$$

$$|A|^2 = \left(Ci(\sin(k\Delta x)) \pm i\sqrt{C^2 \sin^2(k\Delta x) - 1} \right) \left(-Ci(\sin(k\Delta x)) \right. \\ \left. \pm -i\sqrt{C^2 \sin^2(k\Delta x) - 1} \right)$$

$$|A|^2 = \left(Ci(\sin(k\Delta x)) \pm i\sqrt{C^2 \sin^2(k\Delta x) - 1} \right) \left(-Ci(\sin(k\Delta x)) \right. \\ \left. \pm -i\sqrt{C^2 \sin^2(k\Delta x) - 1} \right)$$



$$|A|^2 = i \left(C(\sin(k\Delta x)) \pm \sqrt{C^2 \sin^2(k\Delta x) - 1} \right) \left(-i \left(C(\sin(k\Delta x)) \pm \sqrt{C^2 \sin^2(k\Delta x) - 1} \right) \right)$$
$$|A_{\pm}|^2 = \left(C(\sin(k\Delta x)) \pm \sqrt{C^2 \sin^2(k\Delta x) - 1} \right)^2$$

Existe pelo menos uma raiz em que $|A_{\pm}| > 1$, portanto, a solução é instável $C > 1$



2. $C \leq 1$ e $C(\sin(k\Delta x)) \leq 1$. As duas raízes são:

$$A = Ci(\sin(k\Delta x)) \pm i\sqrt{C^2 \sin^2(k\Delta x) - 1}$$

$$A = Ci(\sin(k\Delta x)) \pm \sqrt{1 - C^2 \sin^2(k\Delta x)}$$

$$|A_+|^2 = \left(Ci(\sin(k\Delta x)) + \sqrt{1 - C^2 \sin^2(k\Delta x)} \right) \left(-Ci(\sin(k\Delta x)) + \sqrt{1 - C^2 \sin^2(k\Delta x)} \right)$$

$$\begin{aligned} |A_+|^2 = & \left(-i^2 C^2 \sin^2(k\Delta x) + Ci(\sin(k\Delta x)) * \left(\sqrt{1 - C^2 \sin^2(k\Delta x)} \right) - Ci(\sin(k\Delta x)) \right. \\ & \left. * \left(\sqrt{1 - C^2 \sin^2(k\Delta x)} \right) + 1 - C^2 \sin^2(k\Delta x) \right) \end{aligned}$$

$$|A_+|^2 = (C^2 \sin^2(k\Delta x) + 1 - C^2 \sin^2(k\Delta x))$$

$$|A_+|^2 = 1$$



Da mesma forma para $|A_-|^2$

$$|A_+|^2 = (C^2 \sin^2(k\Delta x) + 1 - C^2 \sin^2(k\Delta x))$$

$$|A_+|^2 = 1$$

$\therefore$ A condição de estabilidade é $\left| C = \frac{u\Delta t}{\Delta x} \right| \leq 1$.



Modo computacional de CTCS



Esquema leap-frog: duas soluções coexistem

$$u_j^{n+1} = u_j^{n-1} + 2\Delta t g(u_j^n)$$

A equação quadrática $\lambda^2 - 2i\omega\Delta t\lambda - 1 = 0$ possui duas raízes:

$$\lambda_1 \rightarrow 1 \text{ quando } \Delta t \rightarrow 0$$

Modo físico (solução correta)

+

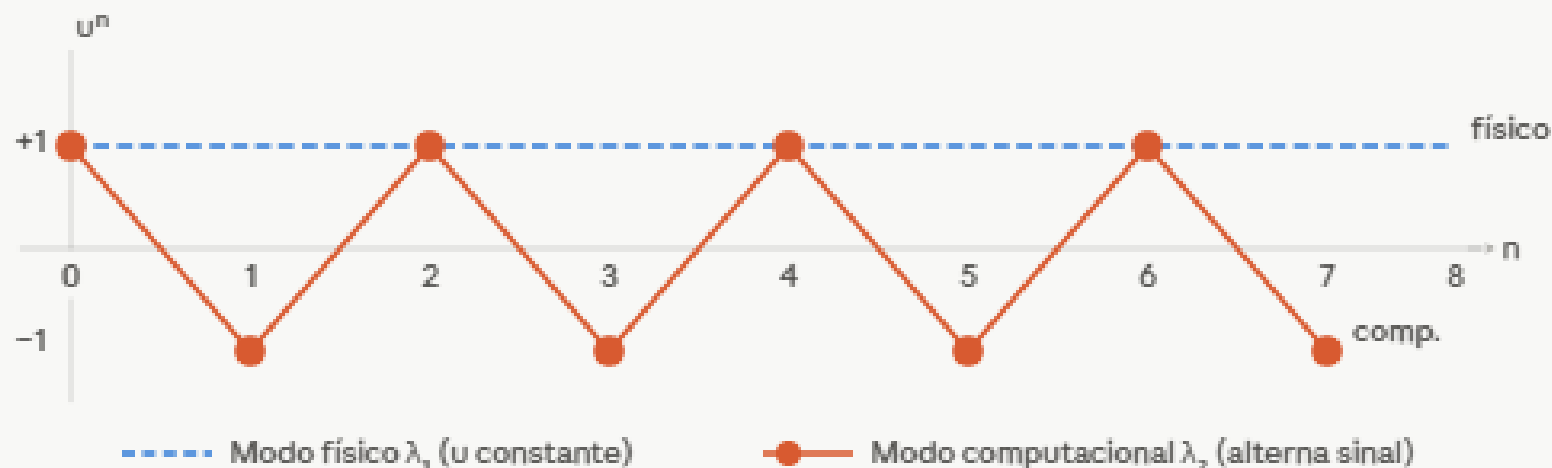
$$\lambda_2 \rightarrow -1 \text{ quando } \Delta t \rightarrow 0$$

Modo computacional (espúrio)

$$\text{Solução geral: } u^n = a \cdot \lambda_1^n \cdot u_1^0 + b \cdot \lambda_2^n \cdot u_2^0$$

Comportamento do modo computacional ($\omega=0$)

Equação $du/dt=0$ integrada com leap-frog, condição: $u^0=1, u^{-1}=-1$



$$\frac{\partial u}{\partial t} = g(u),$$

$$\frac{u_j^{n+1} - u_j^{n-1}}{2\Delta t} = g(u_j^n)$$



Modo computacional de CTCS



Por que o leap-frog gera dois modos?

O esquema leap-frog é um **esquema de três níveis temporais** — ele usa u^{n-1} , u^n e u^{n+1} simultaneamente.

Para integrá-lo você precisa fornecer *duas* condições iniciais: o valor físico $u^{n=0}$ e um valor computacional $u^{n=-1}$.

Considere a **equação de oscilação simples**

$$\frac{\partial u}{\partial t} = i\omega u$$

$$u = u(t)$$

$$\frac{\partial u}{\partial t} = g(u),$$

A **solução analítica exata** é $u(t) = u_0 e^{i\omega t}$, que representa uma **oscilação pura de frequência ω** com amplitude constante $|u| = |u_0|$.



Modo computacional de CTCS



Discretizando com o esquema leap-frog

O **esquema leap-frog** substitui a derivada temporal por uma **diferença centrada de três níveis**:

$$\frac{du}{dt} \approx \frac{u^{n+1} - u^{n-1}}{2\Delta t}$$

$$\frac{\partial u}{\partial t} = i\omega u$$

Aplicando isso à equação de oscilação:

$$\frac{u^{n+1} - u^{n-1}}{2\Delta t} = i\omega u^n$$

Isolando u^{n+1} :

$$\boxed{u^{n+1} = u^{n-1} + 2i\omega\Delta t u^n}$$



Modo computacional de CTCS



Assumindo uma solução na forma de onda

Para analisar o comportamento do esquema, **supomos que a solução discreta** tem a forma:

$$u^n = \lambda^n u_0$$

onde λ é o **fator de amplificação** (um número complexo a determinar) e u_0 é a condição inicial. Isso é a análise de von Neumann aplicada ao tempo.

A ideia:

- Se $|\lambda| = 1$, a amplitude se conserva.
- Se $|\lambda| > 1$, a solução cresce (instável).
- Se $|\lambda| < 1$, decai.



Modo computacional de CTCS



Substituindo $u^n = \lambda^n u_0$ na equação discreta:

$$\lambda^{n+1} u_0 = \lambda^{n-1} u_0 + 2i\omega\Delta t \lambda^n u_0$$

Dividindo tudo por $\lambda^{n-1} u_0$ (assumindo $\lambda \neq 0$ e $u_0 \neq 0$):

$$\lambda^2 = 1 + 2i\omega\Delta t \lambda$$

Reorganizando:

$$\boxed{\lambda^2 - 2i\omega\Delta t \lambda - 1 = 0}$$



Modo computacional de CTCS



Resolvendo a equação quadrática

Usando a fórmula de Bhaskara com $a = 1$, $b = -2i\omega\Delta t$, $c = -1$:

$$\lambda = \frac{2i\omega\Delta t \pm \sqrt{(-2i\omega\Delta t)^2 + 4}}{2}$$

Calculando o discriminante:

$$\Delta = (-2i\omega\Delta t)^2 + 4 = 4(i\omega\Delta t)^2 + 4 = 4[(i\omega\Delta t)^2 + 1]$$

Lembrando que $i^2 = -1$:

$$(i\omega\Delta t)^2 = i^2(\omega\Delta t)^2 = -(\omega\Delta t)^2$$

Portanto:



Modo computacional de CTCS



Resolvendo a equação quadrática

Portanto:

$$\Delta = 4[1 - (\omega\Delta t)^2]$$

$$\sqrt{\Delta} = 2\sqrt{1 - (\omega\Delta t)^2}$$

Substituindo:

$$\lambda_{1,2} = \frac{2i\omega\Delta t \pm 2\sqrt{1 - (\omega\Delta t)^2}}{2}$$

$$\lambda_{1,2} = i\omega\Delta t \pm \sqrt{1 - (\omega\Delta t)^2}$$



Modo computacional de CTCS



Interpretando as duas raízes

O que acontece quando $\Delta t \rightarrow 0$?

$$\lambda_{1,2} = i\omega\Delta t \pm \sqrt{1 - (\omega\Delta t)^2}$$

Expandindo taylor $\sqrt{1 - (\omega\Delta t)^2} \approx 1 - \frac{(\omega\Delta t)^2}{2} + \dots$ para $\omega\Delta t \ll 1$:

Raiz λ_1 (sinal +):

$$\lambda_1 = i\omega\Delta t + \sqrt{1 - (\omega\Delta t)^2} \approx 1 + i\omega\Delta t - \frac{(\omega\Delta t)^2}{2} + \dots$$

Compare com a expansão da solução exata $e^{i\omega\Delta t} \approx 1 + i\omega\Delta t - \frac{(\omega\Delta t)^2}{2} + \dots$

Portanto $\lambda_1 \rightarrow e^{i\omega\Delta t}$ quando $\Delta t \rightarrow 0$, e em particular $\lambda_1 \rightarrow 1$ quando $\Delta t \rightarrow 0$.

Esta é a **raiz física** — ela aproxima a solução verdadeira.

$$f'(x = \Delta x) = (1 - \Delta x^2)^{\frac{1}{2}} \approx \frac{1}{2}(1 - \Delta x^2)^{\frac{1}{2}-1}(-2\Delta x)$$

$$f'(x = \Delta x) = (1 - \Delta x^2)^{\frac{1}{2}} \approx \frac{1}{2}(1 - \Delta x^2)^{-\frac{1}{2}}(-2\Delta x)$$

$$f'(x = \Delta x) = (1 - \Delta x^2)^{\frac{1}{2}} \approx \frac{(-2\Delta x)}{2(1 - \Delta x^2)^{\frac{1}{2}}}$$

$$f'(x = \Delta x) = (1 - \Delta x^2)^{\frac{1}{2}} \approx \frac{(-\Delta x)}{(1 - \Delta x^2)^{\frac{1}{2}}}$$



Modo computacional de CTCS



Raiz λ_2 sinal –:

$$\lambda_{1,2} = i\omega\Delta t \pm \sqrt{1 - (\omega\Delta t)^2}$$

$$\lambda_2 = i\omega\Delta t - \sqrt{1 - (\omega\Delta t)^2} \approx i\omega\Delta t - 1 + \dots$$

Quando $\Delta t \rightarrow 0$: $\lambda_2 \rightarrow -1$.

$$f'(x = \Delta x) = (1 - \Delta x^2)^{\frac{1}{2}} \approx \frac{1}{2}(1 - \Delta x^2)^{\frac{1}{2}-1}(-2\Delta x)$$

$$f'(x = \Delta x) = (1 - \Delta x^2)^{\frac{1}{2}} \approx \frac{1}{2}(1 - \Delta x^2)^{-\frac{1}{2}}(-2\Delta x)$$

$$f'(x = \Delta x) = (1 - \Delta x^2)^{\frac{1}{2}} \approx \frac{(-2\Delta x)}{2(1 - \Delta x^2)^{\frac{1}{2}}}$$

$$f'(x = \Delta x) = (1 - \Delta x^2)^{\frac{1}{2}} \approx \frac{(-\Delta x)}{(1 - \Delta x^2)^{\frac{1}{2}}}$$

Isso significa que $u_2^n = \lambda_2^n u_0 \approx (-1)^n u_0$ —a solução **alterna de sinal a cada passo de tempo**.

Ela **não tem correspondência com nenhuma solução** da equação diferencial original.

É o **modo computacional**, um artefato puro do esquema de três níveis.



Modo computacional de CTCS



Por que $|\lambda_{1,2}| = 1$?

Para o leap-frog aplicado à equação de **oscilação pura**, ambas as raízes têm módulo exatamente 1 quando $\omega\Delta t \leq 1$. Verifique:.

$$|\lambda_{1,2}|^2 = (\omega\Delta t)^2 + \left(\sqrt{1 - (\omega\Delta t)^2}\right)^2 = (\omega\Delta t)^2 + 1 - (\omega\Delta t)^2 = 1 \quad \checkmark$$

Isso significa que o leap-frog é **neutro** — não amortece nem amplifica nenhum dos dois modos.

O modo computacional, uma vez excitado, persiste indefinidamente com amplitude constante.

É por isso que o filtro de Asselin é necessário: para introduzir um amortecimento seletivo sobre o modo de período $2\Delta t$.



Modo computacional de CTCS



$$\lambda_{1,2} = i\omega\Delta t \pm \sqrt{1 - (\omega\Delta t)^2}$$

A raiz λ_1 converge para 1 quando $\Delta t \rightarrow 0$ — é o **modo físico**, a boa.

A raiz λ_2 converge para -1 quando $\Delta t \rightarrow 0$ — é o **modo computacional**, um artefato numérico que **alterna de sinal a cada passo de tempo**.

A solução geral é portanto:

$$u^n = a\lambda_1^n u_1^0 + b\lambda_2^n u_2^0$$

O coeficiente b depende da condição inicial computacional $u^{n=-1}$:

se ela for *exatamente* a correta, $b = 0$ e o modo computacional nunca aparece.

Na prática, qualquer erro na inicialização excita o modo computacional.



Modo computacional de CTCS



Seção 5.1.1 — A condição inicial computacional

A estratégia padrão é calcular $u^{n=-1}$ usando um único passo Euler-forward (que é instável em geral, mas *um* passo não "explode"):

$$u^{n=-1} = u^{n=0} + i\omega\Delta t u^{n=0}$$

Atribuir simplesmente $u^{n=-1} = u^{n=0}$ ativa imediatamente o modo computacional

(caso $\omega = 0$).



Modo computacional de CTCS



O Filtro de Robert-Asselin

A supressão mais comum nos modelos atmosféricos e oceânicos é o **filtro de Robert-Asselin** (Robert 1966, Asselin 1972).

O algoritmo completo a cada passo é:

Leap-frog para obter u^{n+1} :

$$u^{n+1} = u_f^{n-1} + 2\Delta t g(u_f^n)$$

Aplicar o filtro ao nível n (suavização temporal entre três níveis):

$$u_f^n = u^n + \gamma(u_f^{n-1} - 2u^n + u^{n+1})$$

O índice f indica valores filtrados e γ é o **coeficiente de Asselin**, tipicamente entre 0,01 e 0,2.

O filtro funciona como um **smoother temporal discreto** que amortece o modo de período $2\Delta t$ (o modo computacional) pelo fator $(1 - 4\gamma \sin^2(\omega \Delta t / 2))$ a cada passo.



Modo computacional de CTCS



O filtro funciona como um **smoother temporal discreto** que amortece o modo de período $2\Delta t$ (o modo computacional) pelo fator $(1 - 4\gamma \sin^2(\omega\Delta t/2))$ a cada passo.

Efeito colateral importante: o filtro RA torna o critério de estabilidade mais restrito — em vez de $\mu \leq 1$, exige $\mu \leq 1 - \gamma$ (ou equivalentemente $\mu + \gamma \leq 1$).



Filtro de Robert-Asselin-Williams (RAW)

O problema do filtro RA é que ele não conserva a média do campo, degradando a precisão numérica. Williams (2009) corrigiu isso com uma etapa extra que redistribui a correção:

$$u^{n+1} = u_{ff}^{n-1} + 2\Delta t g(u_f^n)$$

$$u_{ff}^n = u_f^n + \frac{\gamma\alpha}{2} (u^{n+1} - 2u_f^n + u_{ff}^{n-1})$$

$$u_f^{n+1} = u^{n+1} - \frac{\gamma(1-\alpha)}{2} (u^{n+1} - 2u_f^n + u_{ff}^{n-1})$$

Para $\alpha = 1$ recuperamos o filtro RA clássico.

Williams (2009) mostrou que $\alpha \approx 0,53$ **minimiza os impactos espúrios** sobre a solução física.



Modo computacional de CTCS





Modo computacional de CTCS



Conceito fundamental: velocidade de fase e de grupo

A **solução exata** da equação de advecção $\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0$ é uma onda que se propaga sem deformar:

$$u(x, t) = u_0 e^{ik(x-ct)} = u_0 e^{i(kx-ckt)} = u_0 e^{i(kx-\omega t)}$$

Onde k é o número de onda e c a velocidade de fase **analítica** (a mesma para todas as ondas — sem dispersão).

A **velocidade de fase** de uma onda $e^{i(kx-\omega t)}$ é:

$$c = \frac{\omega}{k}$$

$$\omega = ck$$

A **velocidade de grupo** (velocidade com que um pacote de ondas se propaga) é:

$$c_g = \frac{d\omega}{dk} = c$$

Para a equação de advecção analítica, $\omega = ck$

então $c = c_g$ — não há dispersão.

O objetivo é mostrar que a **discretização rompe essa igualdade**.



Modo computacional de CTCS



A Velocidade de Fase Computacional

Dispersão numérica — as ondas numéricas propagam com velocidades diferentes das analíticas.

A equação discretizada no espaço. Partimos da equação de advecção com diferença centrada **apenas no espaço** (tempo ainda contínuo):

Para a equação de advecção com diferença centrada no espaço:

$$\frac{\partial u}{\partial t} + c \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = 0$$

Inserindo a **solução de onda** $u_j(t) = u_0 e^{ik(j\Delta x - C_D t)}$, obtém-se a **velocidade de fase computacional**:

$$C_d = c \frac{\sin(k\Delta x)}{k\Delta x}$$

onde C_D é a **velocidade de fase computacional** — o que queremos descobrir.

Note que $j\Delta x$ é a posição espacial e $C_D t$ o deslocamento temporal.



Modo computacional de CTCS



Calculando cada termo:

Derivada temporal (contínua):

$$\frac{\partial u_j}{\partial t} = u_0 (-ikC_D) e^{ik(j\Delta x - C_D t)} = -ikC_D u_j$$

Diferença centrada no espaço:

$$u_{j+1} = u_0 e^{ik((j+1)\Delta x - C_D t)} = u_j e^{ik\Delta x}$$

$$u_{j-1} = u_0 e^{ik((j-1)\Delta x - C_D t)} = u_j e^{-ik\Delta x}$$

Portanto:

$$\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = \frac{u_j e^{ik\Delta x} - u_j e^{-ik\Delta x}}{2\Delta x} = u_j \frac{e^{ik\Delta x} - e^{-ik\Delta x}}{2\Delta x} = u_j \frac{2i \sin(k\Delta x)}{2\Delta x} = u_j \frac{i \sin(k\Delta x)}{\Delta x}$$



Modo computacional de CTCS



Substituindo na equação

$$\frac{\partial u}{\partial t} + c \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = 0$$

$$ikC_D u_j + cu_j \frac{i \sin(k\Delta x)}{\Delta x} = 0$$

Dividindo por $i u_j$ (não-nulo):

$$-kC_D + \frac{c \sin(k\Delta x)}{\Delta x} = 0$$

Isolando C_D :

$$C_D = c \frac{\sin(k\Delta x)}{k\Delta x}$$



Modo computacional de CTCS



Velocidade de grupo computacional

Para calcular C_{Dg} , usamos :

$$\omega_D = kC_D = c \frac{\sin(k\Delta x)}{\Delta x}$$

$$CDg = \frac{d\omega_D}{dk} = \frac{d}{dk} c \frac{\sin(k\Delta x)}{\Delta x} = \frac{c}{\Delta x} \cdot \cos(k\Delta x) \cdot \Delta x = c \cos(k\Delta x)$$

$$CDg = c \cos(k\Delta x)$$



Modo computacional de CTCS



Interpretação física

Normalizando pela velocidade analítica c :

$$\frac{C_D}{c} = \frac{\sin(k\Delta x)}{k\Delta x}$$

$$\frac{CDg}{c} = \cos(k\Delta x)$$

Para ondas longas ($k\Delta x \rightarrow 0$): e — perfeito.

Para ondas curtas ($k\Delta x \rightarrow \pi/2$, ou $\lambda = 4\Delta x$): — a velocidade de grupo **se anula**.

Para ondas ainda mais curtas ($k\Delta x > \pi/2$), C_{Dg} **inverte de sinal** — energia propaga na direção oposta à onda!

Para a onda mais curta possível ($k\Delta x = \pi$, ou $\lambda = 2\Delta x$): — a onda fica **estacionária**.



Modo computacional de CTCS



Dispersão devida à discretização temporal

Passo 1 — A equação discretizada no tempo

Agora discretizamos **apenas no tempo** com leap-frog (espaço contínuo):

(espaço contínuo):

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0$$

$$\frac{u_j^{n+1} - u_j^{n-1}}{2\Delta t} + c \frac{\partial u}{\partial x} = 0$$

Inserindo a solução de onda

$$u_j^n(x) = u_0 e^{ik(x - C_D n \Delta t)}$$

onde $n\Delta t$ é o tempo discreto. Calculando os termos:



Modo computacional de CTCS



Dispersão devida à discretização temporal

onde $n\Delta t$ é o tempo discreto. Calculando os termos:

Diferença centrada no tempo (leap-frog):

$$u^{n+1} = u_0 e^{ik(x-C_D(n+1)\Delta t)} = u^n \cdot e^{-ikC_D\Delta t}$$

$$u^{n-1} = u_0 e^{ik(x-C_D(n-1)\Delta t)} = u^n \cdot e^{+ikC_D\Delta t}$$

Portanto:

$$\frac{u^{n+1} - u^{n-1}}{2\Delta t} = u^n \frac{e^{-ikC_D\Delta t} - e^{ikC_D\Delta t}}{2\Delta t} = u^n \frac{-2i \sin(kC_D\Delta t)}{2\Delta t} = -u^n \frac{i \sin(kC_D\Delta t)}{\Delta t}$$

Derivada espacial (espaço contínuo):

$$c \frac{\partial u^n}{\partial x} = c \cdot (ik) u^n$$



Modo computacional de CTCS



Substituindo e isolando C_D

$$-u^n \frac{i \sin(k C_D \Delta t)}{\Delta t} + c i k u^n = 0$$

Dividindo por $i u^n$:

$$-\frac{\sin(k C_D \Delta t)}{\Delta t} + c k = 0$$

$$\sin(k C_D \Delta t) = c k \Delta t = \omega \Delta t$$

Isolando C_D :

$$\arcsin[\sin(k C_D \Delta t)] = \arcsin[\omega \Delta t]$$

$$(k C_D \Delta t) = \arcsin[\omega \Delta t]$$

$$(C_D) = \frac{\arcsin[\omega \Delta t]}{k \Delta t}$$



Modo computacional de CTCS



Normalizando pela velocidade analítica $c = \omega/k$:

$$\frac{C_D}{c} = \frac{\arcsin[\omega\Delta t]}{kc\Delta t}$$

$$\frac{C_D}{c} = \frac{\arcsin[\omega\Delta t]}{\omega\Delta t}$$

Velocidade de grupo computacional

Para calcular C_{Dg} precisamos de $\omega_D(k)$.

Da relação acima, $\sin(kC_D\Delta t) = \omega\Delta t$, e como $\omega = ck$ para a equação de advecção, temos $\omega\Delta t = ck\Delta t$.

A frequência computacional é então $\omega_D = kC_D$, satisfazendo, ou seja:

$$(C_D) = \frac{\arcsin[\omega\Delta t]}{k\Delta t}$$

$$(kC_D) = \frac{\arcsin[\omega\Delta t]}{\Delta t} = (\omega_D) = \frac{\arcsin[\omega\Delta t]}{\Delta t}$$



Modo computacional de CTCS



A velocidade de grupo computacional:

$$C_{Dg} = \frac{d\omega_D}{dk} = \frac{1}{\Delta t} \frac{d}{dk} \arcsin(ck\Delta t)$$

Usando a regra da cadeia $\frac{d}{dx} \arcsin[f] = \frac{1}{\sqrt{1-f^2}} \frac{df}{dx}$ com $f = ck\Delta t$:

$$C_{Dg} = \frac{1}{\Delta t} \cdot \frac{c\Delta t}{\sqrt{1-(ck\Delta t)^2}} = \frac{c}{\sqrt{1-(ck\Delta t)^2}}$$

Normalizando:

$$\frac{C_{Dg}}{c_g} = \frac{1}{\sqrt{1-(ck\Delta t)^2}}$$



Modo computacional de CTCS



Interpretação física

Para $\omega\Delta t$ pequeno: $C_D/c \rightarrow 1$ e $C_{Dg}/c_g \rightarrow 1$ — bom.

Para $\omega\Delta t \rightarrow 1$ (passo de tempo grande): — a fase computacional é **50% mais rápida** que a analítica!

E $C_{Dg}/c_g \rightarrow \infty$.

Ao contrário da dispersão espacial (que retarda ondas curtas), a dispersão temporal **acelera** todas as ondas — e mais intensamente quanto maior o passo de tempo.



Modo computacional de CTCS



Dispersão espaço-temporal (leap-frog completo)

Este é o caso mais importante: discretização simultânea em espaço e tempo.

O esquema leap-frog completo

$$\frac{u_j^{n+1} - u_j^{n-1}}{2\Delta t} + c \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = 0$$

Inserindo a solução de onda

Tentamos:

$$u_j^n = u_0 e^{ik(j\Delta x - C_D n\Delta t)}$$

Termo temporal (leap-frog):

$$u_j^{n\pm 1} = u_j^n e^{\mp ik C_D \Delta t}$$

$$\frac{u_j^{n+1} - u_j^{n-1}}{2\Delta t} = \frac{u_j^n e^{+ik C_D \Delta t} - u_j^n e^{-ik C_D \Delta t}}{2\Delta t} = u_j^n \frac{e^{+ik C_D \Delta t} - e^{-ik C_D \Delta t}}{2\Delta t} = -u_j^n \frac{i \sin(k C_D \Delta t)}{\Delta t}$$



Modo computacional de CTCS



Dispersão espaço-temporal (leap-frog completo)

Termo espacial (diferença centrada):

$$u_{j\pm 1}^n = u_j^n e^{\pm ik\Delta x}$$

$$\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = \frac{u_j^n e^{+ik\Delta x} - u_j^n e^{-ik\Delta x}}{2\Delta x} = u_j^n \frac{e^{+ik\Delta x} - e^{-ik\Delta x}}{2\Delta x} = u_j^n \frac{i \sin(k\Delta x)}{\Delta x}$$



Modo computacional de CTCS



Substituindo

$$\frac{u_j^{n+1} - u_j^{n-1}}{2\Delta t} + c \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = 0$$

$$-u_j^n \frac{i \sin(k C_D \Delta t)}{\Delta t} + c u_j^n \frac{i \sin(k \Delta x)}{\Delta x} = 0$$

Dividindo por $i u_j^n$: o C_D

$$-\frac{\sin(k C_D \Delta t)}{\Delta t} + c \frac{\sin(k \Delta x)}{\Delta x} = 0$$

$$\sin(k C_D \Delta t) = c \frac{\Delta t}{\Delta x} \sin(k \Delta x) = \mu \sin(k \Delta x)$$

onde $\mu \equiv \frac{c \Delta t}{\Delta x}$ é o **número de Courant**. Isolando C_D :

$$k C_D \Delta t = \arcsin[\mu \sin(k \Delta x)]$$

$$C_D = \frac{\arcsin[\mu \sin(k \Delta x)]}{k \Delta t}$$



Modo computacional de CTCS



Normalizando por $c = \mu\Delta x/\Delta t$:

$$\frac{C_D}{c} = \frac{\arcsin[\mu \sin(k\Delta x)]}{kc\Delta t}$$

$$c\Delta t = \mu\Delta x$$

$$\frac{C_D}{c} = \frac{\arcsin[\mu \sin(k\Delta x)]}{\mu k\Delta x}$$

Velocidade de grupo computacional A frequência computacional é $\omega_D = kC_D$, portanto:

$$kC_D = \frac{\arcsin[\mu \sin(k\Delta x)]}{\Delta t}$$

$$\omega_D = \frac{\arcsin[\mu \sin(k\Delta x)]}{\Delta t}$$



Modo computacional de CTCS



A velocidade de grupo

$$C_{D_g} = \frac{d\omega_D}{dk} = \frac{d}{dk} \left\{ \frac{\arcsin[\mu \sin(k\Delta x)]}{\Delta t} \right\}$$

Usando a regra da cadeia $\frac{d}{dx} \arcsin[f] = \frac{1}{\sqrt{1-f^2}} \frac{df}{dx}$ com $f = \mu \sin(k\Delta x)$:

$$C_{D_g} = \frac{1}{\Delta t} \frac{1}{\sqrt{1 - (\mu \sin(k\Delta x))^2}} \frac{d[\mu \sin(k\Delta x)]}{dk}$$

$$C_{D_g} = \frac{1}{\Delta t} \frac{\mu \cos(k\Delta x) \Delta x}{\sqrt{1 - (\mu \sin(k\Delta x))^2}}$$

Como $\mu \Delta x / \Delta t = c$:

E a razão com a velocidade de grupo analítica $c_g = c$

$$C_{D_g} = c \frac{\cos(k\Delta x)}{\sqrt{1 - (\mu \sin(k\Delta x))^2}}$$

$$\frac{C_{D_g}}{c_g} = \frac{\cos(k\Delta x)}{\sqrt{1 - (\mu \sin(k\Delta x))^2}}$$



Modo computacional de CTCS



Interpretação física dos dois efeitos combinados

Efeito 1 — $\mu \rightarrow 1$: Para $\mu = 1$ exatamente, $\arcsin[\sin(k\Delta x)] = k\Delta x$, portanto $C_D/c = 1$ **para todas as ondas**. O número de Courant unitário cancela completamente a dispersão numérica — resultado notável que justifica usar μ próximo de 1.

Efeito 2 — ondas curtas: À medida que $k\Delta x \rightarrow \pi$ (onda de $2\Delta x$), $\sin(k\Delta x) \rightarrow 0$, logo $C_D \rightarrow 0$ — a onda de dois pontos de grade fica **estacionária**, independentemente de μ .

Efeito 3 — velocidade de grupo negativa: Para $k\Delta x > \pi/2$ e μ pequeno, C_{Dg} pode ser negativa — pacotes de energia propagam na direção **oposta** à da onda física, criando ondas espúrias aparentemente "viajando para trás".



Modo computacional de CTCS



Tabela-resumo das fórmulas deduzidas

Grandeza	Seção 6.1 (só espaço)	Seção 6.2 (só tempo)	Seção 6.3 (leap-frog completo)
Ansatz	$u_j(t) = u_0 e^{ik(j\Delta x - C_D t)}$	$u^n(x) = u_0 e^{ik(x - C_{Dn}\Delta t)}$	$u_j^n = u_0 e^{ik(j\Delta x - C_{Dn}\Delta t)}$
Relação central	$\sin(kC_D\Delta t) \rightarrow -kC_D$	$\sin(kC_D\Delta t) = \omega\Delta t$	$\sin(kC_D\Delta t) = \mu \sin(k\Delta x)$
C_D/c	$\frac{\sin(k\Delta x)}{k\Delta x}$	$\frac{\arcsin(\omega\Delta t)}{\omega\Delta t}$	$\frac{1}{\mu k\Delta x} \arcsin[\mu \sin(k\Delta x)]$
C_{Dg}/c_g	$\cos(k\Delta x)$	$\frac{1}{\sqrt{1 - (\omega\Delta t)^2}}$	$\frac{\cos(k\Delta x)}{\sqrt{1 - [\mu \sin(k\Delta x)]^2}}$
Efeito principal	Retarda ondas curtas	Acelera todas as ondas	Depende de μ

O ponto mais elegante do capítulo inteiro: com $\mu = 1$, $\arcsin[\sin(k\Delta x)] = k\Delta x$, fazendo $C_D/c = 1$ para **toda** escala — os dois erros se cancelam exatamente.

Isso é o que torna o número de Courant unitário tão especial em modelos atmosféricos e oceânicos.



Modo computacional de CTCS



Há dois casos a considerar: o primeiro $C > 1$, e $(C \sin(k\Delta x))^2 > 1$

$$A_{\pm} = C \sin(k\Delta x) \pm \sqrt{C^2 \sin^2(k\Delta x) - 1}$$

$$\geq 0$$

$$A_p = C \sin(k\Delta x) + \sqrt{C^2 \sin^2(k\Delta x) - 1}$$

$$A_c = C \sin(k\Delta x) - \sqrt{C^2 \sin^2(k\Delta x) - 1}$$

$$\phi_j^n = A^n e^{ikj\Delta x}$$

$$\phi_j^n = P A_p^n e^{ikj\Delta x} + C A_c^n e^{ikj\Delta x}$$



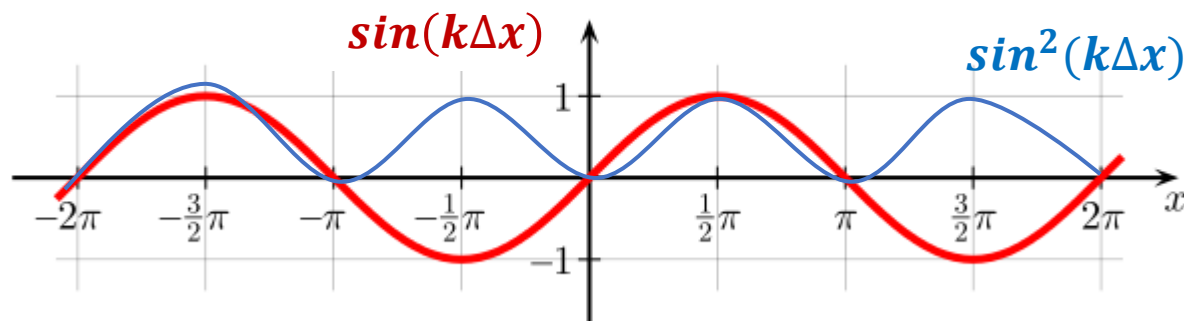
Modo computacional de CTCS



O esquema CTCS dá dois valores para A
 ≥ 0

$$A_p = iC \sin(k\Delta x) + i\sqrt{C^2 \sin^2(k\Delta x) - 1}$$

$$A_c = iC \sin(k\Delta x) - i\sqrt{C^2 \sin^2(k\Delta x) - 1}$$



$$A_p = -ic \sin k\Delta x + (1 - (c \sin k\Delta x)^2)^{1/2} \quad (18)$$

$$A_c = -ic \sin k\Delta x - (1 - (c \sin k\Delta x)^2)^{1/2}, \quad (19)$$



Modo computacional de CTCS



Portanto, a forma geral da solução numérica é

$$\begin{aligned}\phi_j^n &= P A_p^n e^{ikj\Delta x} + C A_c^n e^{ikj\Delta x} \\ \phi_j^n &= [P(A_p)^n + C(A_c)^n] e^{ikj\Delta x}\end{aligned}\tag{20}$$

Vamos escolher $C = 1$. Em seguida, é conveniente escrever A na forma

$$A_p = e^{-i\alpha}, A_c = -e^{i\alpha}$$

Onde $\alpha = k\Delta x = uk\Delta t$ (21)

$$\phi_j^n = P e^{ik(j\Delta x - un\Delta t)} + (-1)^n C e^{ik(j\Delta x + un\Delta t)}.$$

Nos casos em que P e C sejam constantes complexas, determinados pela primeira Condições ou seja, $t=0$ ($n=0$).

$$\phi_j^0 = e^{ikj\Delta x} = (P + C) e^{ikj\Delta x} \Rightarrow (P + C) = 1$$



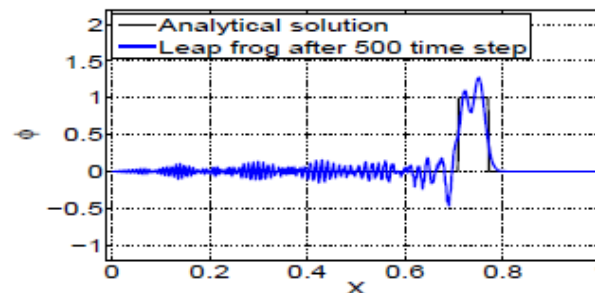
$$\Rightarrow \phi_j^n = \underbrace{(1 - C)e^{ik(j\Delta x - un\Delta t)}}_{\text{Physical mode}} + \underbrace{(-1)^n C e^{ik(j\Delta x + un\Delta t)}}_{\text{Computational mode}} \quad (22)$$

O modo Físico é proporcional à solução **exata** $e^{ik(x-ut)}$

O modo computacional **não correspondem a qualquer solução da equação diferencial original** ; ela é um artefato do Método numérico.

Duas das características do modo computacional

- Ela oscila no tempo de um passo tempo para o próximo passo de tempo (por causa do fator $(-1)^N$)
- Se propaga na direção oposta à verdadeira solução (Por causa do termo $+un\Delta t$ na exponencial ao invés de $-Un\Delta t$).





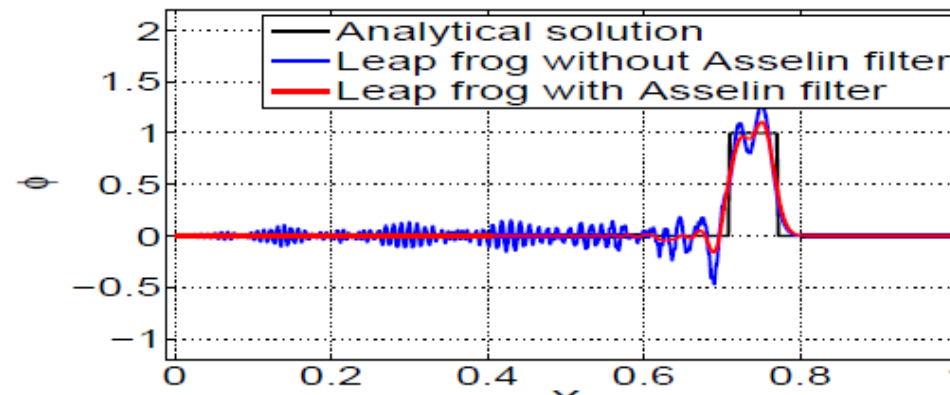
Modo computacional e o filtro RA

- A solução ϕ_j^{n+1} , Depende, ϕ_j^{n-1} Mas não no ϕ_j^n
- Exceto no momento inicial, a solução é encontrada em dois conjuntos de pontos que não são associados.
- Em qualquer ponto J a solução oscila entre os duas soluções Desatrelada.
- Para manter a amplitude do modo computacional pequenas é Necessário soluções acopladas sobre os dois conjuntos alternando de Pontos de grade.
- A maneira mais comum de se fazer isto no modelo Atmosférica é através do Uso do filtro de Robert-Asselin (RA), que fortemente descarrega as Oscilação $\phi^{n+1} = \phi^{n-1} - [\text{other terms}]$
- Calcular o deslocamento do filtro ou seja
$$d = \alpha * (\phi^{n-1} - 2\phi^n + \phi^{n+1})$$
- Aplicar o filtro para Φ^N Ou seja
$$\hat{\phi}^n = \phi^n + d = (1 - 2\alpha)\phi^n + \alpha(\phi^{n+1} + \phi^{n-1})$$

Então, atualize $\hat{\phi}^{n-1}$ por $\hat{\phi}^n$ e $\hat{\phi}^n$ por ϕ^{n+1} Para a próxima iteração
Depois de aplicar o filtro, o esquema de advecção fica parecido com

$$\phi^{n+1} = \hat{\phi}^{n-1} - [\text{other terms}]$$

A figura abaixo mostra advecção de uma onda quadrada (com $C=0,7$) após 500 tempo passo com (linha vermelha) e sem (azul Linha) apresentando Asselin filtro de tempo.



Note que este filtro também induz a um amortecimento artificial do modo físico, α Devem ser mantidos pequenos (na parcela acima $\alpha = 0,05$)



filtro RAW - Williams (2009), MWR



Método RAW, que devemos aplicar a filtragem não somente em Φ^N mas também em Φ^{n+1}

Tal como antes, use primeiro o esquema **leapfrog** como antes

$$\phi^{n+1} = \phi^{n-1} - [\text{other terms}]$$

Em seguida, aplica-se o filtro

$$\hat{\phi}^n = \phi^n + d\beta$$

$$\hat{\phi}^{n+1} = \phi^{n+1} + d(\beta - 1)$$

Depois da implementação do filtro e depois atualizar

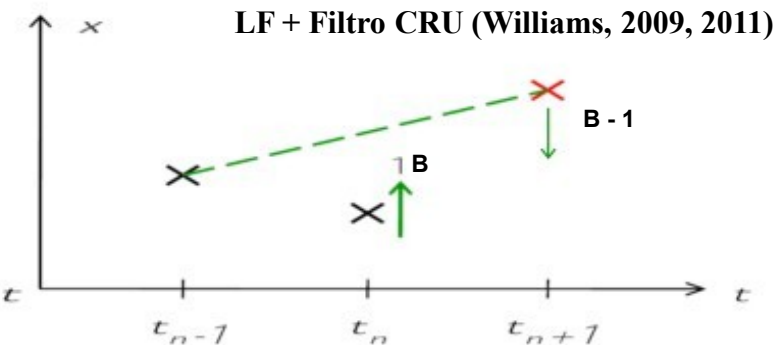
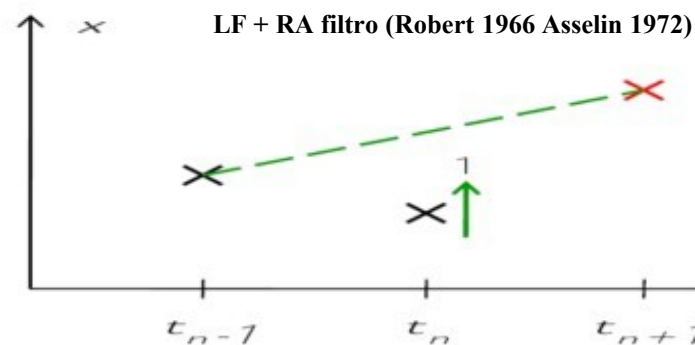
$$\begin{array}{ccc} \hat{\phi}^{n-1} & \Rightarrow & \hat{\phi}^n \\ \hat{\phi}^n & \Rightarrow & \hat{\phi}^{n+1} \end{array}$$

$$\phi^{n+1} = \hat{\phi}^{n-1} - [\text{other terms}]$$

Devido à estabilidade razão , o valor beta é $0,5 < \beta \leq 1$



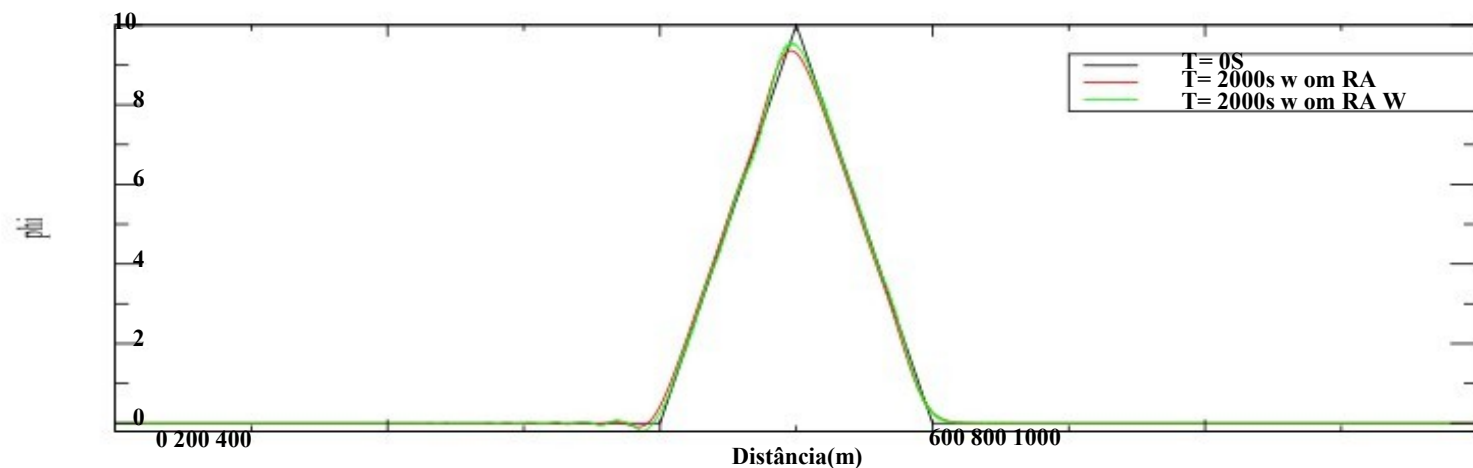
O filtro RA vs RAW



- RA filtro aplica-se em
- Reduz curvatura mas não conserva a curvatura média
- Precisão da amplitude é 1.º ordem

- Filtro-RWA aplica-se em x^n e x^{n+1}
- Reduz e ao mesmo tempo, conserva a curvatura média (para $B=1/2$)
- Precisão da amplitude é $\approx 3.^a$ ordem

Salto com Veto RA_and_RAW_filter social e $dt=0,5$, $\alpha=0,05$, $\beta=0,6$





Modo computacional de CTCS



Filtro de Robert-Asselin

Passo 1 — leap-frog:

$$u^{\wedge}(n+1) = u_f^{\wedge}(n-1) + 2\Delta t \cdot g(u_f^{\wedge}n)$$

Passo 2 — filtro (suavização temporal):

$$u_f^{\wedge}n = u^{\wedge}n + \gamma (u_f^{\wedge}(n-1) - 2u^{\wedge}n + u^{\wedge}(n+1))$$

γ (coeficiente de Asselin) típico: $0,01 \leq \gamma \leq 0,2$

Pequeno: pouca filtragem (modo comp. persiste) | Grande: amortece o modo físico

Efeito na estabilidade

Sem filtro

Estável se $\mu \leq 1$
($\mu = c\Delta t/\Delta x$, Courant)

Com filtro RA

Estável se $\mu \leq 1 - \gamma$
(critério mais restrito!)

Filtro de Robert-Asselin-Williams (RAW) — Williams 2009

$$u^{\wedge}(n+1) = u_ff^{\wedge}(n-1) + 2\Delta t \cdot g(u_f^{\wedge}n)$$

$$u_ff^{\wedge}n = u_f^{\wedge}n + (\gamma\alpha/2)(u^{\wedge}(n+1) - 2u_f^{\wedge}n + u_ff^{\wedge}(n-1))$$

$$u_f^{\wedge}(n+1) = u^{\wedge}(n+1) - \gamma(1-\alpha)/2 \cdot (u^{\wedge}(n+1) - 2u_f^{\wedge}n + u_ff^{\wedge}(n-1))$$

$\alpha = 0,53$ minimiza impactos espúrios | $\alpha=1 \rightarrow$ filtro RA clássico

$$\frac{\partial u}{\partial t} = g(u),$$



Exercício

- Resolver a equação advecção 1D numericamente no domínio $0 \leq X \leq 1000M$. Deixe $\Delta x = 1 M$, e assuma as condições limite periódicas. Suponha que a velocidade de advecção $U = 1 M/s$. Deixe o estado inicial ser um triângulo

$$\Phi(x,0) = \begin{cases} 0 & \text{Para } X < 400 \\ 0.1 (X - 400) & \text{Para } 400 \leq X \leq 500 \\ 20 - 0.1 (x - 400) & \text{Para } 500 \leq X \leq 600 \\ 0 & \text{Para } X > 600 \end{cases}$$

Escolha o intervalo de tempo, que o sistema seja estável. Integre Para 2000seg usando os seguintes esquemas, e mostrar as soluções para $T = 0S$ $T = 200s$, $T = 400S$ $T = 600S$ $T = 800S$ $T = 1000S$ $T = 1200S$ $T = 1400S$ $T = 1600S$ $T = 1800S$ $T = 2000S$.

1. Leap frog com a RA esquema filtro de tempo (use $\alpha = 0,25$)
2. Leap frog com matérias-primas regime filtro de tempo($\beta = 0,60$ e $\beta = 1,0$))



```

MODULE Class_Fields
  IMPLICIT NONE
  PRIVATE
  INTEGER, PUBLIC      , PARAMETER :: r8=8
  INTEGER, PUBLIC      , PARAMETER :: r4=4
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_P(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_C(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_M(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: Deslc(:)
  REAL (KIND=r8),PUBLIC      :: Uvel
  INTEGER      ,PUBLIC      :: iMax
  REAL (KIND=r8),PUBLIC      :: alfa
  PUBLIC :: Init_Class_Fields

```

CONTAINS

```

!-----
-----
SUBROUTINE Init_Class_Fields(xdim,Uvel0,alfa_in)
  IMPLICIT NONE
  INTEGER      , INTENT (IN ) :: xdim
  REAL (KIND=r8), INTENT (IN ):: Uvel0
  REAL (KIND=r8), INTENT (IN ):: alfa_in
  iMax=xdim
  Uvel=Uvel0
  alfa=alfa_in
  ALLOCATE (PHI_P(-1:iMax+2))
  ALLOCATE (PHI_C(-1:iMax+2))
  ALLOCATE (PHI_M(-1:iMax+2))
  ALLOCATE (Deslc(-1:iMax+2))
END SUBROUTINE Init_Class_Fields

```

```

!-----
-----
END MODULE Class_Fields
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

```

```

MODULE Class_NumericalMethod
  USE Class_Fields, Only : PHI_P,PHI_C,PHI_M,Deslc,Uvel,iMax,alfa
  IMPLICIT NONE
  PRIVATE
  INTEGER, PUBLIC      , PARAMETER :: r8=8
  INTEGER, PUBLIC      , PARAMETER :: r4=4
  REAL (KIND=r8) :: Dt
  REAL (KIND=r8) :: Dx
  PUBLIC :: InitNumericalScheme
  PUBLIC :: SchemeForward
  PUBLIC :: SchemeUpdate
  PUBLIC :: SchemeUpStream
  PUBLIC :: Filter_RA

```

CONTAINS

```

!-----
SUBROUTINE InitNumericalScheme(dt_in,dx_in)
  IMPLICIT NONE
  REAL (KIND=r8), INTENT (IN ) :: dt_in
  REAL (KIND=r8), INTENT (IN ) :: dx_in
  INTEGER :: i
  Dt=dt_in
  Dx=dx_in
  DO i=-1,iMax+2
    IF (i*dx < 400.0) THEN
      PHI_C(i)= 0.0
    ELSE IF ( i*dx >= 400.0 .AND. i*dx <= 500.0 ) THEN
      PHI_C(i)= 0.1*(i*dx -400.0)
    ELSE IF( i*dx >= 500.0 .AND. i*dx <= 600.0 )THEN
      PHI_C(i)= 20.0 - 0.1*(i*dx -400.0)
    ELSE IF( i*dx > 600.0 )THEN
      PHI_C(i)= 0.0
    END IF
  END DO
  PHI_M=PHI_C
  PHI_P=PHI_C
END SUBROUTINE InitNumericalScheme

```



```
!-----  
FUNCTION SchemeForward() RESULT(ok)  
  IMPLICIT NONE  
  ! Utilizando a diferenciacao forward  
  !  
  !  $F(j,n+1) - F(j,n)$      $F(j+1,n) - F(j,n)$   
  ! ----- + u ----- = 0  
  !    dt                dx  
  !  
  INTEGER :: ok  
  INTEGER :: j  
  DO j=1,iMax  
    PHI_P(j) = PHI_C(j) - (Uvel*Dt/Dx)*(PHI_C(j+1)-PHI_C(j))  
  END DO  
  CALL UpdateBoundaryLayer()  
END FUNCTION SchemeForward
```

```
!-----  
FUNCTION SchemeUpStream() RESULT (ok)  
  IMPLICIT NONE  
  ! Utilizando a diferenciacao forward no tempo e  
  ! backward no espaco (upstream)  
  !  
  !  $F(j,n+1) - F(j,n)$      $F(j,n) - F(j-1,n)$   
  ! ----- + u ----- = 0  
  !    dt                dx  
  !  
  INTEGER :: ok  
  INTEGER :: j  
  DO j=1,iMax  
    PHI_P(j) = PHI_C(j) - (Uvel*Dt/Dx)*(PHI_C(j)-PHI_C(j-1))  
  END DO  
  CALL UpdateBoundaryLayer()  
END FUNCTION SchemeUpStream
```

```
!-----  
FUNCTION Filter_RA() RESULT (ok)  
  IMPLICIT NONE  
  INTEGER :: i  
  INTEGER :: ok  
  DO i=-1,iMax+2  
    Deslc(i) = alfa*(PHI_M(i) - 2.0*PHI_C(i) + PHI_P(i) )  
    PHI_C(i) = PHI_C(i) + Deslc(i)  
  END DO  
  ok=0  
END FUNCTION Filter_RA
```

```
!-----  
SUBROUTINE UpdateBoundaryLayer()  
  IMPLICIT NONE  
  PHI_P(0) = PHI_P(iMax )  
  PHI_P(-1) = PHI_P(iMax-1)  
  PHI_P(imax+1) = PHI_P(1)  
  PHI_P(iMax+2) = PHI_P(2)  
END SUBROUTINE UpdateBoundaryLayer
```

```
!-----  
FUNCTION SchemeUpdate() RESULT (ok)  
  IMPLICIT NONE  
  INTEGER :: ok  
  PHI_M=PHI_C  
  PHI_C=PHI_P  
  ok=0  
END FUNCTION SchemeUpdate  
!-----  
END MODULE Class_NumericalMethod
```




```

FUNCTION SchemeWriteCtl(nrec) RESULT (ok)
  IMPLICIT NONE
  INTEGER, INTENT (IN) :: nrec
  INTEGER :: ok

  OPEN(UnitCtl,FILE=TRIM(FileName)//'.ctl',FORM='FORMATTED', &
ACCESS='SEQUENTIAL',STATUS='UNKNOWN',ACTION='WRITE')
  WRITE (UnitCtl,'(A6,A      )')'dset ^',TRIM(FileName)//'.bin'
  WRITE (UnitCtl,'(A      )')'title EDO'
  WRITE (UnitCtl,'(A      )')'undef -9999.9'
  WRITE (UnitCtl,'(A6,I8,A18 )')'xdef ',iMax,' linear 0.00 0.001'
  WRITE (UnitCtl,'(A      )')'ydef 1 linear -1.27 1'
  WRITE (UnitCtl,'(A6,I6,A25 )')'tdef ',nrec,' linear 00z01jan0001 1hr'
  WRITE (UnitCtl,'(A20      )')'zdef 1 levels 1000 '
  WRITE (UnitCtl,'(A      )')'vars 1'
  WRITE (UnitCtl,'(A      )')'phic 0 99 resultado da edol yc'
  WRITE (UnitCtl,'(A      )')'endvars'
  CLOSE (UnitCtl, STATUS='KEEP')
  CLOSE (UnitData, STATUS='KEEP')
  ok=0
END FUNCTION SchemeWriteCtl

END MODULE Class_WritetoGrads

```



```
PROGRAM Main
USE Class_Fields, Only :Init_Class_Fields
USE Class_NumericalMethod, Only: InitNumericalScheme, &
  SchemeForward, SchemeUpdate, SchemeUpStream, Filter_RA
USE Class_WritetoGrads, Only :InitClass_WritetoGrads, &
  SchemeWriteData, SchemeWriteCtl
IMPLICIT NONE
INTEGER           , PARAMETER :: r8=8
INTEGER           , PARAMETER :: r4=4
INTEGER           , PARAMETER :: xdim=1000
REAL (KIND=r8)    , PARAMETER :: Uvel0=10.0!m/s
REAL (KIND=r8)    , PARAMETER :: dt=0.08 !s
REAL (KIND=r8)    , PARAMETER :: dx=1.0 !m
INTEGER           , PARAMETER :: ninteraction=400
REAL (KIND=r8)    , PARAMETER :: alfa=0.05

CALL Init()
CALL run()

CONTAINS

SUBROUTINE Init()
  IMPLICIT NONE
  CALL Init_Class_Fields(xdim,Uvel0,alfa)
  CALL InitNumericalScheme(dt,dx)
  CALL InitClass_WritetoGrads
END SUBROUTINE Init
```

```
SUBROUTINE Run()
  IMPLICIT NONE
  INTEGER :: test,it,irec
  irec=0
  DO it=1,ninteraction
    PRINT*,it
    test=SchemeUpStream()
    test=Filter_RA()
    test=SchemeWriteData(irec)
    test=SchemeUpdate()
  END DO
  test=SchemeWriteCtl(ninteraction)
END SUBROUTINE Run

SUBROUTINE Finalize()
  IMPLICIT NONE
END SUBROUTINE Finalize
END PROGRAM Main
```



```

MODULE Class_Fields
  IMPLICIT NONE
  PRIVATE
  INTEGER, PUBLIC      , PARAMETER :: r8=8
  INTEGER, PUBLIC      , PARAMETER :: r4=4
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_P(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_C(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_M(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: Deslc(:)
  REAL (KIND=r8),PUBLIC      :: Uvel
  INTEGER      ,PUBLIC      :: iMax
  REAL (KIND=r8),PUBLIC      :: alfa
  REAL (KIND=r8),PUBLIC      :: beta
  PUBLIC :: Init_Class_Fields

```

CONTAINS

```

!-----
SUBROUTINE Init_Class_Fields(xdim,Uvel0,alfa_in,beta_in)
  IMPLICIT NONE
  INTEGER      , INTENT (IN ) :: xdim
  REAL (KIND=r8), INTENT (IN ):: Uvel0
  REAL (KIND=r8), INTENT (IN ):: alfa_in
  REAL (KIND=r8), INTENT (IN ):: beta_in
  iMax=xdim
  Uvel=Uvel0
  alfa=alfa_in
  beta=beta_in
  ALLOCATE (PHI_P(-1:iMax+2))
  ALLOCATE (PHI_C(-1:iMax+2))
  ALLOCATE (PHI_M(-1:iMax+2))
  ALLOCATE (Deslc(-1:iMax+2))
END SUBROUTINE Init_Class_Fields
!-----
END MODULE Class_Fields

```

```

MODULE Class_NumericalMethod
  USE Class_Fields, Only : PHI_P,PHI_C,PHI_M,Deslc,Uvel,iMax,alfa,beta
  IMPLICIT NONE
  PRIVATE
  INTEGER, PUBLIC      , PARAMETER :: r8=8
  INTEGER, PUBLIC      , PARAMETER :: r4=4
  REAL (KIND=r8) :: Dt
  REAL (KIND=r8) :: Dx

```

```

  PUBLIC :: InitNumericalScheme
  PUBLIC :: SchemeForward
  PUBLIC :: SchemeUpdate
  PUBLIC :: SchemeUpStream
  PUBLIC :: Filter_RAW

```

CONTAINS

```

!-----
SUBROUTINE InitNumericalScheme(dt_in,dx_in)
  IMPLICIT NONE
  REAL (KIND=r8), INTENT (IN ) :: dt_in
  REAL (KIND=r8), INTENT (IN ) :: dx_in
  INTEGER :: i
  Dt=dt_in
  Dx=dx_in
  DO i=-1,iMax+2
    IF (i*dx < 400.0) THEN
      PHI_C(i)= 0.0
    ELSE IF ( i*dx >= 400.0 .AND. i*dx <= 500.0 ) THEN
      PHI_C(i)= 0.1*(i*dx -400.0)
    ELSE IF ( i*dx >= 500.0 .AND. i*dx <= 600.0 ) THEN
      PHI_C(i)= 20.0 - 0.1*(i*dx -400.0)
    ELSE IF ( i*dx > 600.0 ) THEN
      PHI_C(i)= 0.0
    END IF
  END DO
  PHI_M=PHI_C
  PHI_P=PHI_C
END SUBROUTINE InitNumericalScheme
!-----

```



```
FUNCTION SchemeForward() RESULT(ok)
```

```
IMPLICIT NONE
```

```
! Utilizando a diferenciacao forward
```

```
!
```

```
!  $F(j,n+1) - F(j,n)$      $F(j+1,n) - F(j,n)$ 
```

```
! ----- + u ----- = 0
```

```
!     dt                 dx
```

```
!
```

```
INTEGER :: ok
```

```
INTEGER :: j
```

```
DO j=1,iMax
```

```
  PHI_P(j) = PHI_C(j) - (Uvel*Dt/Dx)*(PHI_C(j+1)-PHI_C(j))
```

```
END DO
```

```
CALL UpdateBoundaryLayer()
```

```
END FUNCTION SchemeForward
```

```
!-----
```

```
FUNCTION SchemeUpStream() RESULT (ok)
```

```
IMPLICIT NONE
```

```
! Utilizando a diferenciacao forward no tempo e
```

```
! backward no espaco (upstream)
```

```
!
```

```
!  $F(j,n+1) - F(j,n)$      $F(j,n) - F(j-1,n)$ 
```

```
! ----- + u ----- = 0
```

```
!     dt                 dx
```

```
!
```

```
INTEGER :: ok
```

```
INTEGER :: j
```

```
DO j=1,iMax
```

```
  PHI_P(j) = PHI_C(j) - (Uvel*Dt/Dx)*(PHI_C(j)-PHI_C(j-1))
```

```
END DO
```

```
CALL UpdateBoundaryLayer()
```

```
END FUNCTION SchemeUpStream
```

```
FUNCTION Filter_RAW() RESULT (ok)
```

```
IMPLICIT NONE
```

```
INTEGER :: i
```

```
INTEGER :: ok
```

```
DO i=-1,iMax+2
```

```
  Deslc(i) = alfa*(PHI_M(i) - 2.0*PHI_C(i) + PHI_P(i) )
```

```
  PHI_C(i) = PHI_C(i) + Deslc(i)
```

```
  PHI_P(i) = PHI_P(i) + Deslc(i)*(beta-1.0)
```

```
END DO
```

```
  ok=0
```

```
END FUNCTION Filter_RAW
```

```
!-----
```

```
SUBROUTINE UpdateBoundaryLayer()
```

```
IMPLICIT NONE
```

```
  PHI_P(0)    = PHI_P(iMax )
```

```
  PHI_P(-1)   = PHI_P(iMax-1)
```

```
  PHI_P(imax+1) = PHI_P(1)
```

```
  PHI_P(iMax+2) = PHI_P(2)
```

```
END SUBROUTINE UpdateBoundaryLayer
```

```
!-----
```

```
FUNCTION SchemeUpdate() RESULT (ok)
```

```
IMPLICIT NONE
```

```
INTEGER :: ok
```

```
  PHI_M=PHI_C
```

```
  PHI_C=PHI_P
```

```
  ok=0
```

```
END FUNCTION SchemeUpdate
```

```
!-----
```

```
END MODULE Class_NumericalMethod
```





```
PROGRAM Main
USE Class_Fields, Only :Init_Class_Fields
USE Class_NumericalMethod, Only :InitNumericalScheme, &
  SchemeForward, SchemeUpdate, SchemeUpStream, Filter_RAW
USE Class_WritetoGrads, Only :InitClass_WritetoGrads, &
  SchemeWriteData, SchemeWriteCtl
IMPLICIT NONE
INTEGER          , PARAMETER :: r8=8
INTEGER          , PARAMETER :: r4=4
INTEGER          , PARAMETER :: xdim=1000
REAL (KIND=r8)   , PARAMETER :: Uvel0=10.0 !m/s
REAL (KIND=r8)   , PARAMETER :: dt=0.08    !s
REAL (KIND=r8)   , PARAMETER :: dx=1.0    !m
INTEGER          , PARAMETER :: ninteraction=400
REAL (KIND=r8)   , PARAMETER :: alfa=0.05
REAL (KIND=r8)   , PARAMETER :: beta=0.05 !0.5 < beta <= 1

CALL Init()
CALL run()
```

CONTAINS

```
SUBROUTINE Init()
IMPLICIT NONE
CALL Init_Class_Fields(xdim,Uvel0,alfa,beta)
CALL InitNumericalScheme(dt,dx)
CALL InitClass_WritetoGrads
END SUBROUTINE Init
```

```
SUBROUTINE Run()
IMPLICIT NONE
INTEGER :: test,it,irec
irec=0
DO it=1,ninteraction
  PRINT*,it
  test=SchemeUpStream()
  test=Filter_RAW()
  test=SchemeWriteData(irec)
  test=SchemeUpdate()
END DO
test=SchemeWriteCtl(ninteraction)
END SUBROUTINE Run

SUBROUTINE Finalize()
IMPLICIT NONE

END SUBROUTINE Finalize

END PROGRAM Main
```